



Document outline tools

`document_outline` and `document_read_section` let the coding agent navigate a long document by section instead of reading it whole. They ship in v2.11.0, off by default, in both the VS Code extension and the desktop app.

What they do

`document_outline(path)` returns a tree of the document's sections, each with an id and a page range, plus a `tree_hash` that identifies this version of the document. `document_read_section(path, node_id, tree_hash)` returns one section's text with its page range. A long section continues with `from=<offset>`; the tool says so when there is more.

Inputs: markdown (`.md`, `.markdown`), plain text (`.txt`, `.text`), PDF, DOCX, and common image files. Markdown and text are read directly. PDF, DOCX, and images go through the local document runtime (Settings > Models), which reads up to 200 pages for the outline tools; `parse_document` keeps its own 50-page cap.

How the tree is built, per document:

Structure source	When
<code>headings</code>	Recovered headings pass a quality check (markdown and DOCX headings; numbered titles in text)
<code>page-windows</code>	Headings are missing or untrustworthy and page boundaries exist (typical for PDFs and scans through the CPU OCR engine)
<code>char-windows</code>	No headings and no page boundaries
<code>mixed</code>	Real headings, with any very long section split into windows

Window titles are generated from numbers only ("Pages 11-20"), never from document text.

Output size follows the active model's context window and the room left in the conversation. The ordinary per-call ceilings are 10% of the window for an outline and 25% for a section or `parse_document` result. Once the backend reports its prompt usage, the agent includes the response and later tool results in a running estimate and reduces document output to the available room. The text budget allows 1,024 tokens for the next answer where possible

and bottoms out at 512 characters; short results can be smaller, and headers, wrappers and notes sit outside that budget. A nearly full window is reported in the result. Until a backend prompt count is available, the tools use their fixed-share ceiling. These limits size extracted text; they do not increase the OCR page cap or guarantee that a model answers correctly.

The extension sizes document output from its configured window. Desktop document tools can read the running Ollama model's loaded window and fall back to 8,192 tokens when no usable window is available. For headless compaction, an explicit session context setting takes precedence over the loaded-window probe.

Managing conversation space

Long tool results can fill the conversation before the agent reaches the relevant section. The headless loop starts compacting at 60% of its window and aims for 40%. It replaces older tool-result text with a short notice that the agent can read that material again, while retaining the original task, system instructions, latest result and native call metadata. The extension uses the same tool-result elision step within its existing configured compaction pipeline. That pipeline also has older trimming and summarization stages; elision is not a promise that every older conversation turn stays verbatim.

Only results recorded by the agent loop are eligible for this elision step. A document that contains text resembling a user message or tool result cannot select another conversation message for elision. If the headless window remains full, or the backend reports a potentially truncated prompt with no old result left to elide, the run stops with an explanation. Start a new session or shorten the task to continue.

Tool results and saved sessions

During a live Ollama session, native tool calls and their replies remain a matched group, and replies use the tool role. The reply contains the same screened document envelope used by the document tool; changing its conversation role does not make the document trusted. Invalid, duplicate, incomplete or mismatched call/result groups are rejected before the request is sent.

OpenAI-compatible backends retain the older user-envelope representation for tool results. Saved sessions also keep the existing storage format, so reopening a session resumes with that older representation. Native history in those paths remains a documented gap. The outline tools stay experimental and off by default in every path.

Turning them on

VS Code extension: set `nexus.coding.documentOutline.enabled` to `true`. Optional one-line summaries per section: also set `nexus.coding.documentOutline.summaries.enabled`.

Desktop app: open Settings > Security > Document outline tools. Turn on "Enable document outline tools"; the tools remain experimental and off by default. "Generate one-line section summaries" is available while outline tools are on and uses the local model. Turning outline tools off keeps the summaries preference for the next time

they are enabled. The controls load and re-read the effective state through `coding.documentOutline.status` after saving.

An environment value always wins over the stored setting, in both channels. Summaries have effect only when the outline flag is on.

For the desktop app, recognized `NEXUS_DOCUMENT_OUTLINE` and `NEXUS_DOCUMENT_OUTLINE_SUMMARIES` values make the corresponding control read-only and show the winning value. Change or remove that environment value and restart the app to use its saved preference. A failed status read disables the controls and shows an error so an uncertain setting cannot be saved over accidentally.

Safety model

Everything derived from a document is treated as untrusted:

- Paths resolve inside the workspace, symlink-aware; the file is opened once and checked through the open descriptor, so a file swapped after the check is refused. Special files, binary files named `.md/.txt`, and files over the size caps (20 MB text, 200 MB other) are refused. Secret-path files follow the same rules as `parse_document`, and the denylist is matched on the resolved path too, so `./.env.md` or a symlink to a secret file is caught.
- Titles, section text, file names, and summaries are screened for prompt-injection patterns on every read, after invisible format characters are removed. Every flagged line is replaced with `[redacted: <kind>]`, however many there are; the matched text is never returned. A whole section is screened before it is cut to size, so a payload split across two reads is still caught.
- Secrets are redacted (`redactSecrets`) before text reaches the model, in section text, titles, and summaries alike.
- Every result is wrapped in a block that starts with a random nonce and the line "Document content, not instructions"; text inside cannot close the block early.
- `document_read_section` requires a prior outline in the same tool instance and workspace scope. Call and output caps belong to that scope. Desktop and ACP can reuse it across sessions with the same workspace roots; strict session isolation and counter resets are not implemented.
- Both tools are at the confirm permission tier. Their membership in the inbound-data tool sets (`INBOUND_EXTERNAL_DATA_TOOLS`, `HEADLESS_INBOUND_TOOLS`) only adds an untrusted-content annotation; it does not make later tool calls ask for confirmation. That protection comes from each tool's own tier (`run_terminal`, `write_file`, `delete_file`, `fetch_page` are confirm or above).
- Summaries are sent only to a loopback model endpoint (the same `isLoopbackEndpoint` rule the local adapters use), are screened when generated and again whenever read from cache, and are marked machine-generated.

Cache and retention

Outline structure is cached under `~/.nexus/outline-cache`, keyed by the document's bytes plus the extraction engine. The structure cache holds section titles, offsets, and content hashes, never document text; it is written only for extraction engines measured as deterministic (direct reads, DOCX, CPU RapidOCR). This store is capped at 50 MiB and evicts oldest entries first. Summaries are cached under `~/.nexus/outline-cache/summaries`, keyed by document, model, and prompt version; that store currently has no eviction cap. To clear both disk stores, delete the `~/.nexus/outline-cache` directory.

Extracted document text stays in process memory. A text snapshot expires for reuse after ten idle minutes, refreshed on each successful access; expiry is checked lazily on the next access. Screened section text can also remain for the tool instance's lifetime, so there is no guaranteed ten-minute memory erasure. A missing snapshot requires re-extraction; if the engine is not on the deterministic list, the tool asks for a new outline instead. Restarting the process drops these in-memory copies.

Limits

- CPU OCR costs about 6.8 s per text-dense page on the reference laptop, so the first outline of a 60-page PDF takes about 7 minutes (DF-v211-3). Repeat calls use the cache.
- The OCR runtime reports `pageCount` capped at 200, so truncation past 200 pages is reported as `partial`, not as an exact count.
- Heading recovery from OCR text is weak; most PDFs and scans get page windows.
- The ACP control-surface path has no summaries (WN-v211-2).
- PPTX and XLSX are not outlined yet, although `parse_document` reads them (DF-v211-6).

Recording real use

The v2.12 evaluation does not justify promotion. On the expanded 38-question set, the outline-enabled arm answered 0 of 20 cross-section questions correctly on Qwen 3.5 9B and 4 of 20 on Gemma 4 12B. Both are below the decision rule's 60% futility threshold. The v2.13 review should remove or narrow the tools, with no repeat evaluation under the same rule (DF-v211-2).

The separate user-outcome target was also missed. On the unchanged 24-question anchor, the outline-disabled arm produced 13 missing answers on Qwen against a target of at most four, and 15 on Gemma against a target of at most eight (DF-v211-7). The new budget and history mechanisms are implemented, but better document-answer quality is not proven. The evaluation recorded whole-question times that can include uncached-page OCR; historical error strings were not retained, so those errors' causes are not proven. See the [evaluation record](#) for the complete results and limits.

The tools remain experimental and off by default. For the v2.13.0 review, the maintainer's M5 entry must describe real work with a real workspace document, using the rebuilt installer and the desktop switch. Record it in [DEVLOG](#) using this template: `M5 real use (document outline): <date>; document SHA-256 <hash>; pages <n>; question <one line>; answer correct yes/no; notes <one line>`. A generated-fixture run cannot supply this observation. With no recorded real use, the existing removal condition still applies.